역할 분담 개정안

ChatApp · 미니게임과 일정 공유를 담은 채팅 앱

13조 | 김지원 · 이정윤 | 2026.11.04

김지원: 프론트엔드(Swing 클라이언트)
이정윤: 백엔드(Java TCP 소켓 서버)

기능별로 서버와 화면을 함께 맡던 기존 방식을 변경합니다. 김지원은 전체 화면과 클라이언트 통신을, 이정윤은 서버 처리와 데이터 관리를 담당합니다.

담당 범위

영역	김지원 · 프론트엔드	이정윤 · 백엔드
기반 구조	전체 화면 틀, 공통 색상·폰트·버튼, 화면 이동	서버 실행, 연결 수락, 클라이언트별 스레드, 공유 상태 관리
소켓 통신	서버 연결·종료, 요청 전송, 수신 스레드, 응답에 따른 화면 갱신	메시지 해석·분배, 요청 검증, 처리 결과·이벤트 전송
채팅	접속·로비·채팅 화면, 방 목록·참가자 표시, 입력·귓속말 UI	닉네임 중복 검사, 접속자·방 관리, 방별 채팅·귓속말 전달
미니게임	요청·수락·선택 화면, 대기 상태와 결과 표시	참가자·게임 상태 관리, 선택 수집, 승패 판정, 중도 이탈 처리
이모티콘	보유 목록·선택 창, 채팅 표시, 보상 획득 안내	보상 지급, 중복 지급 방지, 보유 목록 관리, 사용 권한 검사
일정	날짜별 조회·등록·수정·삭제 화면, 입력 검증	방별 일정 관리, 변경 권한 검사, 변경 공유, 저장·복원 범위 구현
예외 처리	접속 실패·끊김·서버 오류 안내, 중복 클릭·전송 방지	잘못된 요청, 비정상 종료, 동시 접근, 게임 중 퇴장 처리

공동으로 결정하고 확인할 일

- JDK·빌드 방식, 메시지 형식, 사용자·방·게임·일정 ID, 오류 코드와 이벤트 전달 범위를 합의합니다.
- 미니게임은 **2인 가위바위보 1종을 권장**합니다. 무승부·시간 초과·퇴장 시 처리와 보상 기준은 함께 확정합니다.
- 서버 재시작 후 유지할 데이터와 사용자 식별 방식을 정합니다. 저장 기능은 백엔드가 맡고 저장 범위는 공동 결정합니다.
- 매주 실제 연결 테스트를 진행하고, 각자 담당 구조를 설명할 수 있도록 발표 자료를 준비합니다.

이 문서의 역할 분담은 기존 기능별 분담안을 대체합니다. 프로젝트의 채팅·게임·이모티콘·일정 공유 범위는 유지합니다.

4주 개발 일정

제출일이 정해지지 않아 주차 기준으로 작성했습니다. 실제 마감에 맞춰 조정합니다.

기간	김지원 · 프론트엔드	이정윤 · 백엔드	공동 완료 기준
1주차 기반·채팅	화면 틀과 디자인 통일, 접속·로비·채팅 UI, 소켓 연결·송수신, 방·참가자 표시	서버·스레드 구조, 닉네임 처리, 방 생성·입장·퇴장, 방별 채팅, 접속자 목록	클라이언트 3개 접속, 방 2개에서 메시지 분리, 접속 종료 반영
2주차 게임·보상	게임 요청·선택·결과 UI, 이모티콘 목록·전송, 귓속말 UI 완성	게임 진행·판정, 보상 지급·검증, 이모티콘 전송, 귓속말 처리	게임 승자가 보상을 한 번 받고 채팅에서 사용
3주차 일정·통합	일정 조회·등록·수정·삭제, 입력 오류 안내, 전체 화면 연결	일정 CRUD·권한 검사·변경 공유, 합의한 저장·복원, 전체 기능 연결	같은 방의 일정 변경 동기화, 다른 방과 데이터 분리
4주차 검증·제출	화면 오류·멈춤·중복 입력 개선, 사용 흐름과 시연 화면 정리	동시접속·비정상 종료·예외 처리 검증, 실행 설정·서버 로그 정리	서로 다른 PC에서 전체 시연, 실행 안내·발표 자료·영상 완성

1주차 시작 순서

단계	할 일과 산출물
공동 합의	화면 구성도, 개발 환경, 기본 메시지 요청·응답 예시를 먼저 작성합니다.
병렬 개발	김지원은 임시 데이터로 화면을 만들고, 이정윤은 테스트 클라이언트로 서버를 검증합니다.
첫 연결	닉네임 접속 요청과 성공·실패 응답부터 연결해 실제 통신 경로를 확인합니다.
기능 확장	방 목록 → 생성·입장 → 채팅 → 퇴장·접속 종료 순서로 연결합니다.
통합 확인	클라이언트 3개·방 2개로 확인하고, 실행 방법과 발견한 오류를 공유합니다.

김지원의 화면 구성

접속 화면 → 로비 → 채팅방
채팅방에는 대화창·입력창·참가자 목록을 두고, 게임·일정·이모티콘은 별도 창으로 엽니다. 공통 색상·폰트·여백·버튼 스타일을 한곳에서 관리합니다.

임시 데이터는 화면 개발용으로만 사용합니다. 실제 기능 완료 여부는 서버와 연결한 결과를 기준으로 판단합니다.

통신·Git 협업과 검증

화면과 서버를 독립적으로 개발하되, 작은 기능 단위로 자주 연결합니다.

통신 연결 원칙

• **클라이언트 연결은 하나:** 각 화면이 소켓을 따로 만들지 않고 공통 통신 코드를 통해 요청을 보냅니다. 김지원이 클라이언트 송수신 구조를 담당합니다.

• **서버가 상태 확정:** 게임 승패, 보유 이모티콘, 일정 수정 권한은 이정윤의 서버 코드에서 검증합니다. 화면의 입력 검사만으로 판단하지 않습니다.

• **화면 멈춤 방지:** 네트워크 수신은 별도 스레드에서 수행하고 Swing 화면 갱신은 이벤트 처리 스레드에서 실행합니다.

• **프로토콜 문서 공동 관리:** 메시지 종류·필수 필드·응답·오류·수신 대상을 docs/protocol.md에 기록하고 변경 전에 공유합니다.

예: 일정 창에서 등록 요청 → 공통 통신 코드 → 서버 검증·저장 → 같은 방에 변경 알림 → 각 클라이언트의 일정 화면 갱신

저장소와 담당 경계

경로·브랜치 예시	담당·운영 방식
client/ · feature/client-chat	김지원: 화면과 클라이언트 네트워크 코드
server/ · feature/server-chat	이정윤: 서버 통신·기능 처리·데이터 관리
common/ · docs/protocol.md	공동: 메시지와 공통 데이터 구조. 변경 시 서로 확인
main	검증된 통합 버전. 기능 브랜치 → PR → 상대 검토 → 병합

저장소: github.com/Kim-Ji-Won/ChatApp
지원 작업 폴더: /Users/jiwon/Network_Programming/팀프로젝트
각자의 로컬 저장 위치는 달라도 되며 저장소 내부 구조는 동일하게 유지합니다. 위 경로·브랜치는 합의용 예시입니다.

최종 테스트·시연 체크리스트

검증 항목	완료 기준
접속·채팅방	중복 닉네임 처리, 방별 메시지 분리, 접속자·퇴장 상태 일치
게임·이모티콘	동일한 게임 결과 표시, 보상 1회 지급, 미보유 이모티콘 사용 거부
일정 공유	같은 방에 변경 반영, 수정·삭제 권한 준수, 다른 방 데이터 분리
안정성	연결 끊김 안내, 서버의 접속 정리, 나머지 사용자 이용 지속

발표·제출 담당

김지원: 화면 구조, 클라이언트 송수신·수신 스레드, 화면 갱신, 사용자 시연
이정윤: 서버 스레드, 방·게임 상태, 판정·권한 검사, 저장 기능
공동: 프로토콜·통합 테스트, 실행 안내서, 발표 자료와 영상. 제출 형식은 수업 공지에 맞춥니다.